

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.metrics import accuracy_score, f1_score
```

```
# Step 1: Prepare the Data
# Input data for 2-input logic gates (AND, OR, XOR)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
# Outputs for each logic gate
Y_AND = np.array([[0], [0], [0], [1]]) # AND Gate
Y_OR = np.array([[0], [1], [1], [1]]) # OR Gate
Y_XOR = np.array([[0], [1], [1], [0]]) # XOR Gate
# NOT Gate input (1-input data)
X_NOT = np.array([[0], [1]])
Y_NOT = np.array([[1], [0]])
```

```
# Step 2: Define the Feed-Forward Neural Network Model
def build_ffnn(input_dim):
    model = Sequential()
    # Input Layer and first hidden layer
    model.add(Dense(4, input_dim=input_dim, activation='relu'))
    # Second hidden layer
    model.add(Dense(4, activation='relu'))
    # Output layer
    model.add(Dense(1, activation='sigmoid'))
    # Compile the model
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model

# Build models for each gate
model_and = build_ffnn(2)
model_or = build_ffnn(2)
model_xor = build_ffnn(2)

# Train each model
model_and.fit(X, Y_AND, epochs=100, verbose=0)
model_or.fit(X, Y_OR, epochs=100, verbose=0)
model_xor.fit(X, Y_XOR, epochs=100, verbose=0)

# NOT gate model needs 1 input
model_not = build_ffnn(1)
model_not.fit(X_NOT, Y_NOT, epochs=100, verbose=0)
```

```
<keras.src.callbacks.history.History at 0x7a16cffce4e0>
```

```
# Step 3: Evaluate and Predict Part 1
# AND Gate Evaluation
pred_and = np.round(model_and.predict(X)).astype(int)
print("AND Gate Predictions:")
print(pred_and)
print("Accuracy:", accuracy_score(Y_AND, pred_and))
print("F1 Score:", f1_score(Y_AND, pred_and))
```

```
1/1 ————— 0s 71ms/step
AND Gate Predictions:
[[0]
 [0]
 [0]
 [0]]
Accuracy: 0.75
F1 Score: 0.0
```

```
# OR Gate Evaluation
pred_or = np.round(model_or.predict(X)).astype(int)
print("\nOR Gate Predictions:")
print(pred_or)
print("Accuracy:", accuracy_score(Y_OR, pred_or))
print("F1 Score:", f1_score(Y_OR, pred_or))
```

[Show hidden output](#)

```
# Step 3: Evaluate and Predict Part 2
# XOR Gate Evaluation
```

```
pred_xor = np.round(model_xor.predict(X)).astype(int)
print("\nXOR Gate Predictions:")
print(pred_xor)
print("Accuracy:", accuracy_score(Y_XOR, pred_xor))
print("F1 Score:", f1_score(Y_XOR, pred_xor))
```

1/1  0s 66ms/step

XOR Gate Predictions:

```
[[0]
 [1]
 [1]
 [1]]
```

Accuracy: 0.75

F1 Score: 0.8

```
# NOT Gate Evaluation
pred_not = np.round(model_not.predict(X_NOT)).astype(int)
print("\nNOT Gate Predictions:")
print(pred_not)
print("Accuracy:", accuracy_score(Y_NOT, pred_not))
print("F1 Score:", f1_score(Y_NOT, pred_not))
```

1/1  0s 68ms/step

NOT Gate Predictions:

```
[[1]
 [1]]
```

Accuracy: 0.5

F1 Score: 0.6666666666666666